

Assignment – 8.2

2303A51860

Task 1 – Test-Driven Development for Even/Odd Number Validator

PROMPT :

Generate test cases for a function `is_even(n)` and then implement the function so that it satisfies all generated tests, using assert test with 5 test cases.

CODE :

```
AI-82PY > _
1 #generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests, using assert test with 5 test cases.
2 # Test cases for is_even function
3 def test_is_even():
4     assert is_even(2) == True, "Test case 1 failed"
5     assert is_even(3) == False, "Test case 2 failed"
6     assert is_even(0) == True, "Test case 3 failed"
7     assert is_even(-4) == True, "Test case 4 failed"
8     assert is_even(-5) == False, "Test case 5 failed"
9     print("All test cases passed!")
10 # Implementation of is_even function
11 def is_even(n):
12     return n % 2 == 0
13 # Run the test cases
14 test_is_even()
15
```

OUTPUT :

```
PS D:\AI> & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python' '51402' '--' 'D:\AI\AI-8.2.PY'
All test cases passed!
PS D:\AI>
```

JUSTIFICATION :

This task demonstrates the importance of validating input types before processing logic. By generating test cases first, students ensure the function correctly handles edge cases like zero, negative numbers, and large integers. It reinforces robust error handling and logical correctness. TDD ensures the implementation strictly follows predefined behavior expectations.

Task 2 – Test-Driven Development for String Case Converter

PROMPT :

Generate test cases for two functions: `to_uppercase(text)`, `to_lowercase(text)` it satisfies all generated tests, using assert test with 4 test cases.

CODE :

```
17 #generate test cases for two functions: to_uppercase(text), to_lowercase(text) it satisfies all generated tests, using assert test with 4 test cases.
18 # Test cases for to_uppercase and to_lowercase functions
19 def test_case_conversion():
20     # Test cases for to_uppercase
21     assert to_uppercase("hello") == "HELLO", "Test case 1 failed for to_uppercase"
22     assert to_uppercase("World") == "WORLD", "Test case 2 failed for to_uppercase"
23
24     # Test cases for to_lowercase
25     assert to_lowercase("HELLO") == "hello", "Test case 1 failed for to_lowercase"
26     assert to_lowercase("World") == "world", "Test case 2 failed for to_lowercase"
27
28     print("All test cases passed!")
29
30 # Implementation of to_uppercase and to_lowercase functions
31 def to_uppercase(text):
32     return text.upper()
33 def to_lowercase(text):
34     return text.lower()
35
36 # Run the test cases
37 test_case_conversion()
```

OUTPUT :

```
● PS D:\AI> & 'c:\Users\aatig\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\
'D:\AI\AI-8.2.PY'
All test cases passed!
○ PS D:\AI>
```

JUSTIFICATION :

This task highlights safe string manipulation and input validation practices. AI-generated test cases help identify corner cases such as empty strings and invalid inputs like `None` or numbers. It emphasizes defensive programming and clean function design. Using TDD ensures reliability before deployment.

Task 3 – Test-Driven Development for List Sum Calculator

PROMPT :

Generate test cases for a function `sum_list(numbers)` that calculates the sum of list elements it satisfies all generated tests, using assert test with 4 test cases.

CODE :

```

38 #generate test cases for a function sum_list(numbers)that calculates the sum of list elements it satisfies all generated tests, using assert test with 4 test cases.
39 # Test cases for sum_list function
40 def test_sum_list():
41     assert sum_list([1, 2, 3]) == 6, "Test case 1 failed"
42     assert sum_list([-1, -2, -3]) == -6, "Test case 2 failed"
43     assert sum_list([]) == 0, "Test case 3 failed"
44     assert sum_list([0, 0, 0]) == 0, "Test case 4 failed"
45     print("All test cases passed!")
46 # Implementation of sum_list function
47 def sum_list(numbers):
48     return sum(numbers)
49 # Run the test cases
50 test_sum_list()
51

```

OUTPUT :

```

● PS D:\AI> & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'D:\AI\AI-8.2.PY'
All test cases passed!
○ PS D:\AI>

```

JUSTIFICATION :

This task focuses on data validation and handling heterogeneous lists. By generating tests first, students ensure the function correctly processes empty lists, negative values, and ignores non-numeric elements safely. It strengthens logical filtering and exception handling concepts. TDD guarantees accurate computation under diverse conditions.

Task 4 – Test Cases for Student Result Class

PROMPT :

Generate test cases for a StudentResult class with the following methods: add_marks(mark), calculate_average(), get_result(), it satisfies all generated tests, using assert test with 3 test cases.

CODE :

```

52 #Generate test cases for a StudentResult class with the following methods: add_marks(mark), calculate_average(), get_result(),it satisfies all generated tests, using
53 # Test cases for StudentResult class
54 def test_student_result():
55     student = StudentResult()
56
57     # Test case 1: Adding marks and calculating average
58     student.add_marks(80)
59     student.add_marks(90)
60     assert student.calculate_average() == 85, "Test case 1 failed"
61
62     # Test case 2: Getting result based on average
63     assert student.get_result() == "Pass", "Test case 2 failed"
64
65     # Test case 3: Adding more marks and recalculating average and result
66     student.add_marks(50)
67     assert student.calculate_average() == 73.33, "Test case 3 failed"
68     assert student.get_result() == "Pass", "Test case 3 failed"
69
70     print("All test cases passed!")
71 # Implementation of StudentResult class
72 class StudentResult:
73     def __init__(self):
74         self.marks = []
75
76     def add_marks(self, mark):
77         self.marks.append(mark)
78
79     def calculate_average(self):
80         if not self.marks:
81             return 0
82         return round(sum(self.marks) / len(self.marks), 2)
83
84     def get_result(self):
85         average = self.calculate_average()
86         return "Pass" if average >= 40 else "Fail"
87 # Run the test cases
88 test_student_result()

```

OUTPUT :

```

● PS D:\AI> & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'D:\AI\AI-8.2.PY'
All test cases passed!
○ PS D:\AI>

```

JUSTIFICATION :

This task introduces object-oriented design with validation constraints. AI-generated test cases help verify correctness of mark ranges, average calculation, and pass/fail logic. It promotes structured class-based development and method testing. TDD ensures all class behaviors meet academic result evaluation standards.

Task 5 – Test-Driven Development for Username Validator

PROMPT :

Generate a function that Test-Driven Development for Username Validator, Minimum length: 5 characters, No spaces allowed, Only alphanumeric characters. it satisfies all generated tests, using assert test with 4 test cases.

CODE :

```

92 #generate a function that Test-Driven Development for Username Validator, Minimum length: 5 characters, No spaces allowed, Only alphanumeric characters.
93 # it satisfies all generated tests, using assert test with 4 test cases.
94 # Test cases for username_validator function
95 def test_username_validator():
96     assert username_validator("user1") == True, "Test case 1 failed"
97     assert username_validator("us") == False, "Test case 2 failed"
98     assert username_validator("user name") == False, "Test case 3 failed"
99     assert username_validator("user@name") == False, "Test case 4 failed"
100     print("All test cases passed!")
101 # Implementation of username_validator function
102 def username_validator(username):
103     if len(username) < 5:
104         return False
105     if ' ' in username:
106         return False
107     if not username.isalnum():
108         return False
109     return True
110 # Run the test cases
111 test_username_validator()
112

```

OUTPUT :

```

● PS D:\AI> & 'c:\Users\aatq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatq\AppData\Local\Programs\Python\Python314\python.exe' 'D:\AI\AI-8.2.PY'
All test cases passed!
○ PS D:\AI>

```

JUSTIFICATION :

This task emphasizes input validation and security best practices. AI-generated test cases ensure strict enforcement of length, character restrictions, and formatting rules. It prevents invalid or insecure usernames from being accepted. TDD guarantees the function meets defined validation policies reliably.